

Programação concorrente e paralela

Nome: (preencher) Data: 23/09/2026

Os códigos estão separados por questão: 1-ambiente, 2-io (a sequencial e b asyncio) e 3-cpu (a sequencial, b threads, c processos, d OpenMP, e Cython). Este texto responde as três partes: o ambiente, a aplicação que espera a rede e a aplicação que só calcula.

1. Ambiente de desenvolvimento

Os tempos foram medidos neste notebook, direto no Windows. A descrição da máquina entra no trabalho porque o speedup depende dela: pedir 12 threads só faz sentido depois de dizer quantos núcleos existem e se eles são iguais.

Item	Valor
Máquina	Notebook Lenovo, modelo 21NQ
Processador	Intel Core i5-13420H (13ª geração)
Núcleos físicos	8 (4 de desempenho e 4 de eficiência)
Processadores lógicos	12
Frequência	Base 2,1 GHz; turbo até 4,6 GHz nos núcleos rápidos
Cache	12 MB (L3)
Memória principal	16 GB (o Windows mostra cerca de 15,5 GiB)
Sistema operacional	Windows 11 Pro, 64 bits, versão 10.0.26200
Python	3.13.9, com GIL ativo
Compilador C	GCC 16.2.0, opção -O2, com OpenMP
Máquina virtual	Os testes rodaram no Windows, no próprio notebook

Os 4 núcleos de desempenho têm hyper-thread: cada um aparece como dois processadores lógicos. Os outros 4 são núcleos de eficiência, mais fracos. A conta $4 \times 2 + 4 = 12$. Por isso, lá na frente, o tempo melhora bem até 8 threads e quase para em 12.

Há um Ubuntu 24.04.3 no WSL2, que é uma máquina virtual leve da Microsoft. Dentro dele aparecem 12 CPUs e 7,5 GiB de memória. Esse Ubuntu não foi usado nos tempos: não tinha gcc instalado. O Makefile da pasta serve para compilar o mesmo C nesse Linux, se for preciso.

2. Aplicação IO-bound

IO-bound quer dizer que o programa passa o tempo esperando algo de fora. Aqui, esperando a internet. A aplicação baixa 16 páginas da documentação do Python e soma o tamanho do que chegou. A lista está em `urls.py` e é a mesma nas duas versões.

`io_sequencial.py` usa `urllib`. Pede uma página, espera ela chegar inteira, e só então pede a próxima. `io_asyncio.py` usa `asyncio` com `aiohttp`. Pede as 16 juntas. Enquanto uma ainda não respondeu, o programa já está cuidando das outras. Continua sendo um fluxo só: ele alterna entre as esperas, sem colocar a conta em vários núcleos.

Rodei duas vezes. Nas duas, os dois programas leram 3.410.820 bytes. A tabela abaixo é a segunda rodada. A primeira deu 13,37 s e 0,71 s.

Versão	Bytes lidos	Tempo
Sequencial (urllib)	3410820	13,62 s
Concorrente (asyncio)	3410820	0,70 s
Ganho (sequencial / asyncio)	-	19,4 vezes

O ganho vem da espera. No sequencial, o tempo da página 2 só começa quando a página 1 terminou. No asyncio, as esperas acontecem no mesmo intervalo. Passou de 16 vezes porque o

aihttp também reaproveita a conexão, e o urllib abre uma conexão nova para cada endereço.

3. Aplicação CPU-bound

CPU-bound quer dizer que o tempo vai embora calculando. Esperar junto não adianta. Para ficar mais rápido é preciso mais núcleos, ou um código compilado.

O problema é contar quantos inteiros de 1 até N são abundantes. Um número é abundante quando a soma dos divisores próprios é maior do que ele. O 12 é abundante: $1+2+3+4+6 = 16$. Até 100 existem 22, e usei isso como conferência. Para cada n, o laço testa divisores até a raiz de n. O custo total é $O(N \text{ raiz de } N)$, maior que $O(N)$. Cada n é independente, então o intervalo pode ser repartido.

Usei $N = 500000$. Todas as versões medidas devolveram 123779. A repartição é a mesma: com p trabalhadores, o de índice i fica com i, i+p, i+2p... Assim os números grandes, que custam mais, não ficam todos no último.

Versões medidas:

1. `cpu_sequencial.py`: um laço em Python, do 1 até N.
2. `cpu_threads.py`: `ThreadPoolExecutor`, função em Python. O GIL fica preso, então só uma thread calcula por vez.
3. `cpu_threads_sem_gil.py`: o mesmo `ThreadPoolExecutor`, mas a conta está em `divisores.c` e é chamada por ctypes. Nessa chamada o GIL fica solto.
4. `cpu_processos.py`: `ProcessPoolExecutor`. Cada processo tem o próprio interpretador e o próprio GIL.
5. `openmp.c`: a mesma função em C. A região paralela reparte as threads e `reduction(+:total)` soma as contagens no fim.

A quinta do enunciado, Cython com OpenMP, está em `cydiv.pyx`. Não medi: o Python do Windows pede o compilador da Microsoft para gerar a extensão.

Speedup e eficiência usam o tempo da própria versão com 1 trabalhador: $S(p) = T(1) / T(p)$ e $E(p) = S(p) / p$. Exemplo do OpenMP com 8 threads: $2,061 / 0,281 = 7,335$ e $7,335 / 8 = 0,917$. A coluna de tempo é que compara Python com C. OpenMP e threads sem GIL: mediana de 5 rodadas. As versões em Python: uma rodada.

O que cada versão faz de fato

A conta matemática é a mesma nas cinco versões. O que muda é quem executa: uma pessoa, várias threads, vários processos ou o C. Todas devolveram 123779 abundantes até 500000.

A função `soma_divisores_proprios`, em `3-cpu/abundante.py`, soma os divisores menores que o próprio número. No 12 ela começa com 1. O 2 divide 12, então entram 2 e o par 6. O 3 divide 12, então entram 3 e o par 4. A soma fica $1+2+6+3+4 = 16$. O próximo candidato já passa da raiz de 12, e o laço para. 16 é maior que 12, então 12 é abundante e o contador sobe 1. Parar na raiz evita testar de novo um divisor cujo par já entrou.

`conta_abundantes(inicio, passo, limite)` aplica isso numa fatia. Começa em inicio, pula de passo em passo e para ao passar do limite. Devolve só a contagem, não a lista. Com 4 trabalhadores e limite 12 as fatias são: 1 olha 1, 5, 9; 2 olha 2, 6, 10; 3 olha 3, 7, 11; 4 olha 4, 8, 12. Nessa faixa só o 12 é abundante, e ele cai no trabalhador 4. As contagens 0, 0, 0 e 1 somam 1. Ninguém olhou o mesmo número duas vezes.

a) Sequencial, em `3-cpu/a-sequencial/cpu_sequencial.py`. Chama `conta_abundantes(1, 1, limite)`: olha 1, depois 2, depois 3, até N, na mesma pessoa. Levou 17,3 s.

b) Threads com GIL, em `3-cpu/b-threads/cpu_threads.py`. Cria p threads. Cada uma chama a mesma

função Python, com início 1, 2, 3... e passo p. No fim soma as contagens parciais. O GIL só deixa uma thread executar esse laço por vez, então o tempo ficou perto de 20 s com 2, 4, 8 ou 12 threads.

b) Threads sem GIL, em `cpu_threads_sem_gil.py`. A repartição é a mesma. Quem soma os divisores é a função em `divisores.c`, compilada como `divisores.dll`. Enquanto o C roda, o Python solta o GIL e as threads calculam juntas. Com 1 thread já deu 2,2 s, porque o C compilado faz a mesma conta mais rápido. Com 12 threads deu 0,27 s.

c) Processos, em `3-cpu/c-processos/cpu_processos.py`. As fatias são as mesmas. Cada uma vai para outro Python, com a própria trava, rodando de novo `conta_abundantes`. Com 8 processos deu 5,2 s. Com 1 processo deu 22,7 s, acima do sequencial, porque criar o processo entrou no cronômetro. Com 12 processos voltou a 6,2 s.

d) OpenMP, em `3-cpu/d-openmp/openmp.c`. `omp_set_num_threads` fixa quantas threads nascem. O bloco com `#pragma omp parallel` roda nelas ao mesmo tempo. A primeira thread recebe id 0, soma 1 e começa no número 1. A segunda começa no 2. O passo é a quantidade de threads. Cada uma chama `conta_abundantes` de `divisores.c`. `reduction(+:total)` dá uma cópia do contador para cada thread e soma as cópias no fim. Com 8 threads deu 0,281 s. Com 12 deu 0,255 s.

e) Cython, em `3-cpu/e-cython/cydiv.pyx`. Escreve a mesma soma de divisores e o mesmo teste. O prange percorre de 1 até N e entrega os números às threads, de um em um. `nogil=True` solta a trava, e o `+= 1` soma no final, no papel do `reduction`. Não medi: neste Windows a extensão não compilou. Se rodasse, o resultado teria de ser 123779 e o tempo deveria ficar perto do OpenMP.

Tempos, speedup e eficiência

Sequencial e threads com GIL

Versão	p	Tempo (s)	Speedup	Eficiência
sequencial	1	17,294	1,000	1,000
threads com GIL	1	18,477	1,000	1,000
threads com GIL	2	21,310	0,867	0,434
threads com GIL	4	21,502	0,859	0,215
threads com GIL	8	19,996	0,924	0,116
threads com GIL	12	20,243	0,913	0,076

Threads sem GIL (ctypes + C)

Versão	p	Tempo (s)	Speedup	Eficiência
threads sem GIL	1	2,167	1,000	1,000
threads sem GIL	2	0,863	2,511	1,256
threads sem GIL	4	0,517	4,192	1,048
threads sem GIL	8	0,335	6,466	0,808
threads sem GIL	12	0,275	7,888	0,657

Multiprocessing

Versão	p	Tempo (s)	Speedup	Eficiência
multiprocessing	1	22,701	1,000	1,000
multiprocessing	2	10,451	2,172	1,086
multiprocessing	4	7,821	2,903	0,726
multiprocessing	8	5,230	4,341	0,543
multiprocessing	12	6,242	3,637	0,303

OpenMP

Versão	p	Tempo (s)	Speedup	Eficiência
OpenMP	1	2,061	1,000	1,000

Versão	p	Tempo (s)	Speedup	Eficiência
OpenMP	2	0,839	2,456	1,228
OpenMP	4	0,478	4,312	1,078
OpenMP	8	0,281	7,335	0,917
OpenMP	12	0,255	8,082	0,674

Leitura dos resultados

Com 1 trabalhador, o C já levou 2,06 s e o Python 17,29 s. São cerca de 8 vezes, vindas do compilador, antes do paralelo. Por isso, no gráfico de tempo geral, OpenMP e threads sem GIL ficam colados no eixo. O segundo gráfico abre só essas duas.

Threads com GIL não aceleram. O tempo fica perto de 20 s de 2 até 12 threads, e o speedup fica abaixo de 1. O GIL deixa uma thread executar Python por vez. As outras esperam a trava. A eficiência cai até 0,076 com 12 threads porque o tempo não diminui e o divisor aumenta.

Threads sem GIL e OpenMP descem juntos: cerca de 2,1 s com 1 thread e cerca de 0,26 s com 12. O pacote de threads é o mesmo da versão anterior. A diferença é a conta longa rodar em C com o GIL solto. No OpenMP, com 8 threads o speedup foi 7,3 e a eficiência 0,92. Com 12 threads o tempo quase não muda (0,255 s) e a eficiência cai para 0,67, porque os núcleos extras não são iguais aos primeiros.

Com 2 threads a eficiência passou de 1 (1,23 no OpenMP). O resultado continuou 123779. O clock sobe quando o processador vê mais trabalho (turbo), então a medição com 1 thread fica um pouco mais lenta e o speedup passa levemente de 2.

Multiprocessing acelera porque cada processo tem seu GIL. Com 1 processo deu 22,7 s, acima dos 17,3 s do sequencial: criar processo no Windows entrou no cronômetro. Com 8 processos chegou a 5,23 s. Com 12 voltou a 6,24 s. Processo pesa mais que thread, e 12 já passa do que os núcleos rápidos sustentam.

Juntar as contagens no final é um trecho serial minúsculo. O teto do speedup neste trabalho vem do processador, dos 4 núcleos rápidos e dos 4 fracos, e não de um pedaço grande do programa que tenha ficado de fora do paralelo.

Gráficos

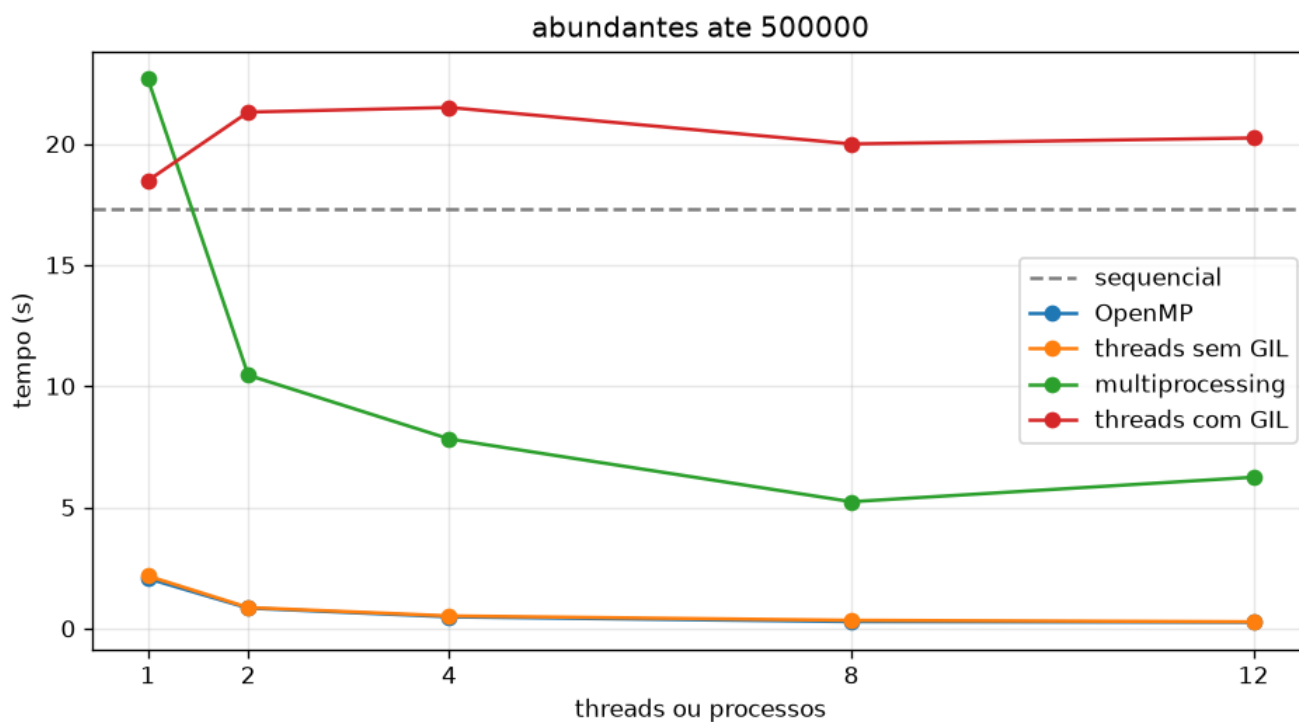


Figura 1. Tempo de execução. A linha tracejada é o Python sequencial.

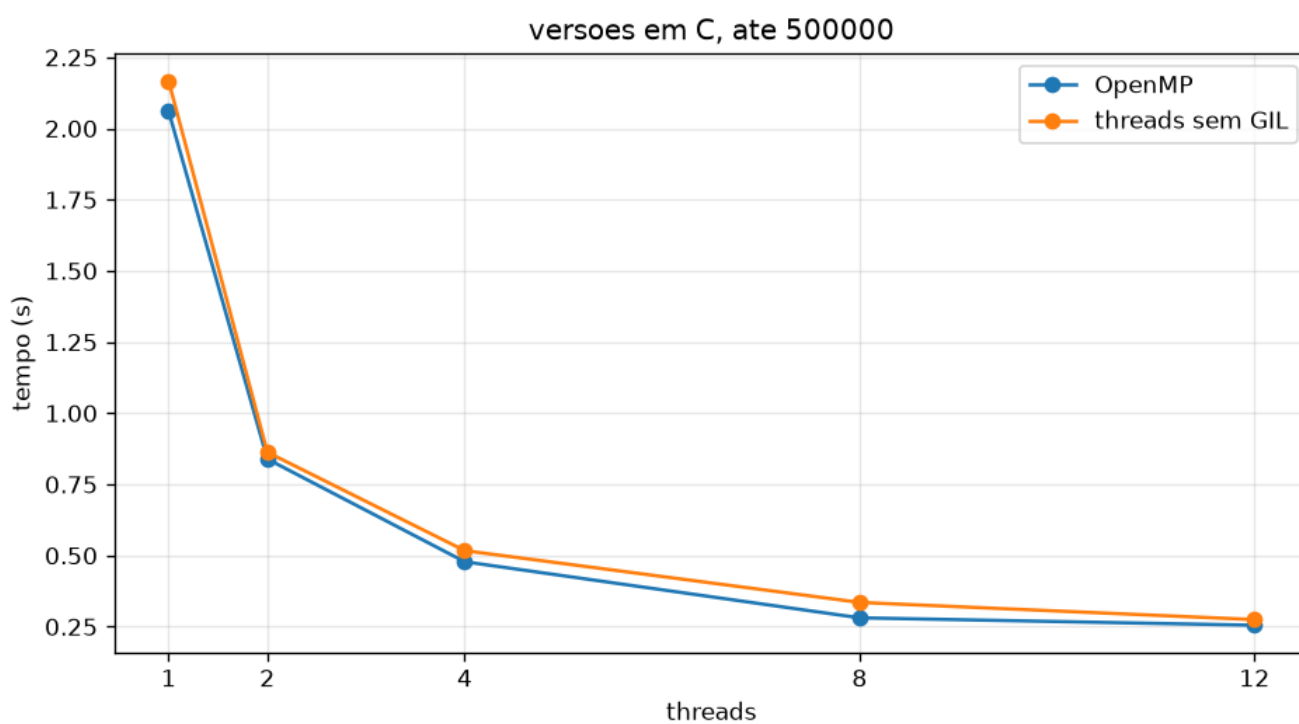


Figura 2. Tempo só das versões em C (OpenMP e threads sem GIL).

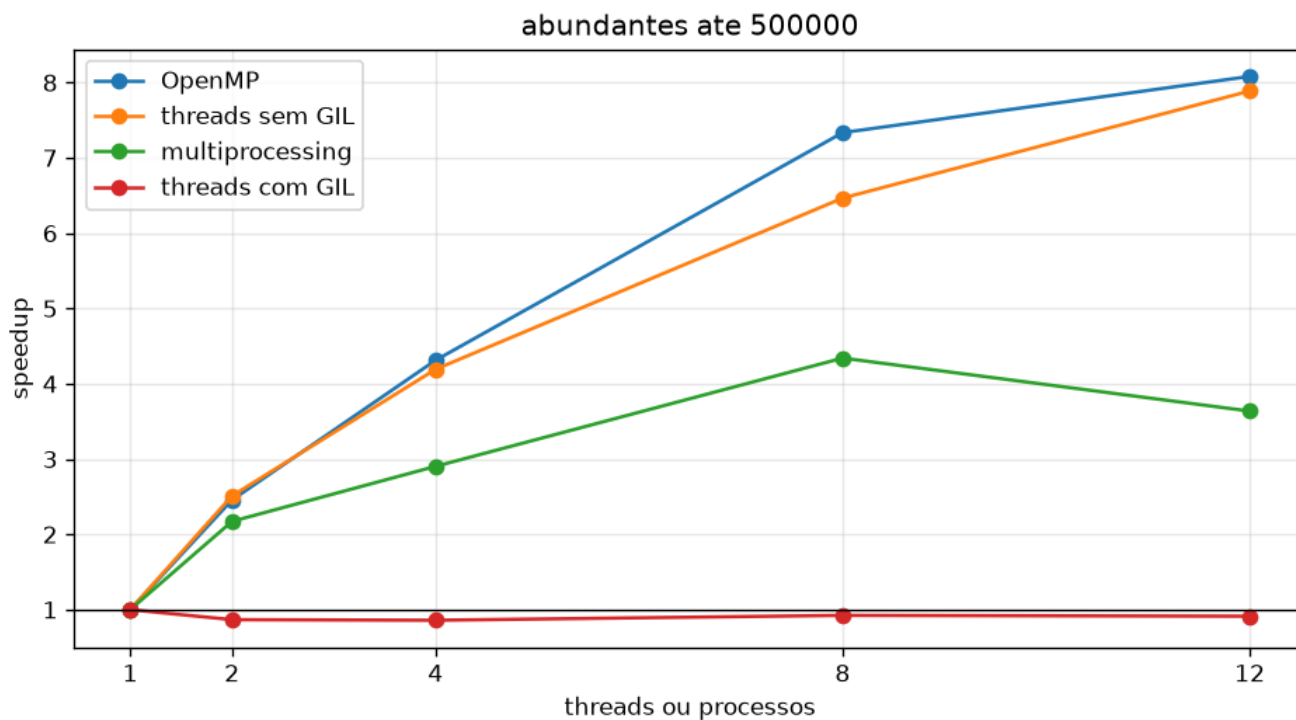


Figura 3. Speedup em relação à mesma versão com 1 trabalhador. A linha preta vale 1.

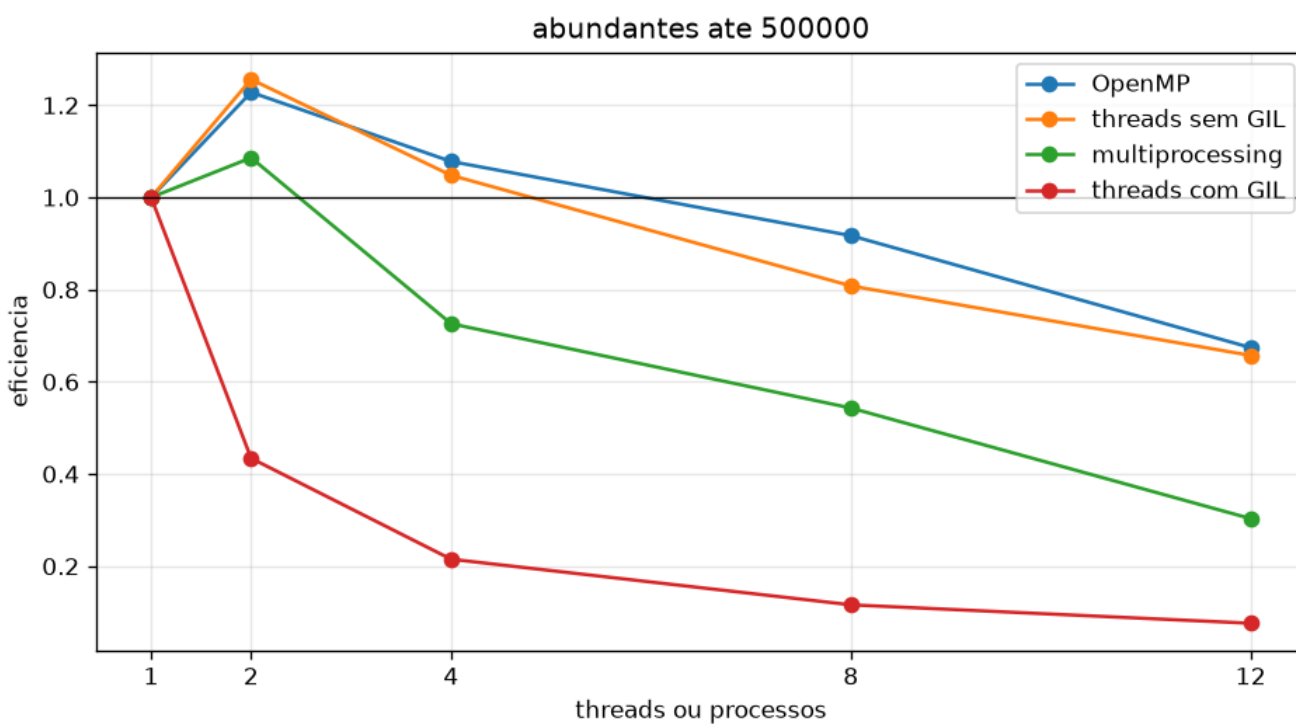


Figura 4. Eficiência (speedup / p). A linha preta vale 1.